2024.1 Multicore Computing

Final Exam (8pm-9:20pm, June 17)

StudentID# : ( ) , Name : ( )

*** Your answers should be written in English.**

1. (35 points) Fill in the blanks (a)~(n) with the most appropriate English words.

- OpenMP is a framework for (a.) memory parallel computing. This means all threads have access to the same, globally (same as (a)), memory in OpenMP programming models.
  OpenMP API consists of three components: (b.), library routines, and (c.).
  - (same as (b)): typed in your source code, these are instructions for the compiler on how to parallelize your code.
  - library routines: typed in your source code, these are function calls to functions of the OpenMP library.
  - (same as (c)): handled in the terminal or OS, these are variables used by the system and/or processes.
- (d.) is a computer graphics rendering technique that generates a result image by tracing the path of light from the view camera (which determines your view into the scene), through the 2D viewing plane (pixel plane), out into the 3D scene, and back to the light sources.
- (e.) provides much higher instruction throughput and memory bandwidth than (f.) within a similar price and power envelope. Many applications leverage these higher capabilities to run faster on (same as (e)) than on (same as (f)). This difference in capabilities between (same as (e)) and (same as (f)) exists because they are designed with different goals in mind. While (same as (f)) is designed to excel at executing a sequence of operations, called a thread, as fast as possible and can execute a few tens of these threads in parallel, (same as (e)) is designed to excel at executing thousands of them in parallel (amortizing the slower single-thread performance to achieve greater throughput). (same as (e)) is specialized for highly parallel computations and therefore designed such that more transistors are devoted to data processing rather than data (g. c) and flow (h. c).
- (i.) is a proprietary compiler by Nvidia intended for use with CUDA. CUDA code runs on both the CPU and GPU. (same as (i)) separates these two parts and sends (j. h) code (the part of code which will be run on the CPU) to a C/C++ compiler like GCC or Intel C++ Compiler (ICC) or Microsoft Visual C++ Compiler, and sends the (k. d) code (the part which will run on the GPU) to the GPU. The (same as (k)) code is further compiled by (same as (i)).
- [In CUDA] threads share data via shared memory within (l.).
- [In OpenMP] By default, all variables declared outside a parallel region are shared variables except (m.).
- [In OpenMP] (n.) is a type of OpenMP schedule that is similar to the dynamic schedule, but the size of the portion of work (i.e. chunk size) decreases as the program runs.

2. (18 points) Following program intends to compute the approximation value of numerical integration $\int_0^1 \frac{4.0}{(1+x^2)} dx$ with multiple number of threads (i.e. 8 threads) using OpenMP.

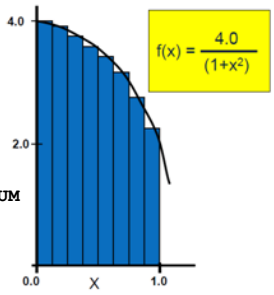
(1) Insert in line 11 appropriate code that specifies the number of threads to use in subsequent parallel regions to be **NUM.**

(2-a) The following program is not correct and may generate a wrong result. Which part in the program is wrong? : line ( )

(2-b) Why is the program wrong? Your answer : **([use less than 20 English words])**

(2-c) How can you modify the program code for correction? **Modify or insert your code directly in the following code.**

```
1:  #include <omp.h>
2:  #include <stdio.h>
3:  #define NUM 8
4:  long num_steps = 10000000;
5:  double step;
6:
7:  void main ()
8:  {
9:        long i; double x, pi, sum = 0.0;
10:       step = 1.0/(double) num_steps;
11:       [                                ]  // specifies the number of threads to use
                                               // in subsequent parallel regions to be NUM
12:       #pragma omp parallel for reduction(+:sum)
13:       for (i=0;i< num_steps; i++){
14:                 x = (i+0.5)*step;
15:                 sum = sum + 4.0/(1.0+x*x);
16:       }
17:       pi = step * sum;
18:       printf("pi=%.4lf\n",pi);
19: }
```



3. (24 points) Consider following CUDA code and its example execution result. Insert appropriate CUDA code in empty boxes below. You must not use for-loop in your code. You should create totally 36 GPU threads with two-dimensional layout of grid and blocks.

```
1: #include <stdio.h>

2: __global__ void exec_conf() {
3:     (a)
4:     (b)
5:     printf("block_index:(%d,%d), thread_index:(%d,%d)
       , array_index:(%d,%d)\n",
       (c)
6: }

7: int main(void) {
8:     (d)                                          ;
9:     (e)                                          ;
10:    printf("##### exec_conf starts! #####\n");
11:    exec_conf (f)                                ;
12:    cudaDeviceSynchronize();
13:    return 0;
14:}
```

Example of Execution Output Result:

```
##### exec_conf starts! #####
block_index:(0,2), thread_index:(0,0), array_index:(0,4)
block_index:(0,2), thread_index:(1,0), array_index:(1,4)
block_index:(0,2), thread_index:(2,0), array_index:(2,4)
block_index:(0,2), thread_index:(0,1), array_index:(0,5)
block_index:(0,2), thread_index:(1,1), array_index:(1,5)
block_index:(0,2), thread_index:(2,1), array_index:(2,5)
block_index:(1,0), thread_index:(0,0), array_index:(3,0)
block_index:(1,0), thread_index:(1,0), array_index:(4,0)
block_index:(1,0), thread_index:(2,0), array_index:(5,0)
block_index:(1,0), thread_index:(0,1), array_index:(3,1)
block_index:(1,0), thread_index:(1,1), array_index:(4,1)
block_index:(1,0), thread_index:(2,1), array_index:(5,1)
block_index:(1,1), thread_index:(0,0), array_index:(3,2)
block_index:(1,1), thread_index:(1,0), array_index:(4,2)
block_index:(1,1), thread_index:(2,0), array_index:(5,2)
block_index:(1,1), thread_index:(0,1), array_index:(3,3)
block_index:(1,1), thread_index:(1,1), array_index:(4,3)
block_index:(1,1), thread_index:(2,1), array_index:(5,3)
block_index:(1,2), thread_index:(0,0), array_index:(3,4)
block_index:(1,2), thread_index:(1,0), array_index:(4,4)
block_index:(1,2), thread_index:(2,0), array_index:(5,4)
block_index:(1,2), thread_index:(0,1), array_index:(3,5)
block_index:(1,2), thread_index:(1,1), array_index:(4,5)
block_index:(1,2), thread_index:(2,1), array_index:(5,5)
block_index:(0,0), thread_index:(0,0), array_index:(0,0)
block_index:(0,0), thread_index:(1,0), array_index:(1,0)
block_index:(0,0), thread_index:(2,0), array_index:(2,0)
block_index:(0,0), thread_index:(0,1), array_index:(0,1)
block_index:(0,0), thread_index:(1,1), array_index:(1,1)
block_index:(0,0), thread_index:(2,1), array_index:(2,1)
block_index:(0,1), thread_index:(0,0), array_index:(0,2)
block_index:(0,1), thread_index:(1,0), array_index:(1,2)
block_index:(0,1), thread_index:(2,0), array_index:(2,2)
block_index:(0,1), thread_index:(0,1), array_index:(0,3)
block_index:(0,1), thread_index:(1,1), array_index:(1,3)
block_index:(0,1), thread_index:(2,1), array_index:(2,3)
```

4. (23 points) You can easily compute the summation of element values of an array in parallel using OpenMP code like "`#progma omp parallel for` **`reduction`** `(+:sum)`". In problem4, you are required to implement the parallel summation using OpenMP without using **`reduction`**. Insert appropriate C/C++ code in the empty box below to complete the implementation of the parallel summation using OpenMP. Note that you must not use "**`reduction`**" clause in your OpenMP code. Instead, you must use a devide-and-conquer approach (i.e. recursive form) to define the **`parallel_sum`** function to efficiently compute the summation in parallel. We do not want to create too many threads. Therefore, we use the third parameter variable **`S`** of the function as the sequential cutoff to control the number of threads to be created. If the size of the input array becomes smaller than **`S`**, the function will not divide any more (i.e. will stop recursion) and the function will perform sequential summation. Your code should utilize parallelism efficiently.

```
int parallel_sum ( int* A, int N, int S )
// A     : starting memory address (pointer) of an input integer array
// N     : the number of elements in the input array for summation, meaning N elements will be added together in this function.
// S     : sequential cutoff. If N becomes smaller than S, the function will stop recursion and perform sequential summation.
// Note that you must not use "reduction" OpenMP clause in this function. Your code should utilize parallelism efficiently.
// Instead, you must use divide-and-conquer approach (i.e. recursive form)
//                    to define and implement this function to efficiently compute the summation in parallel.
{
    // insert your code here to implement parallel summation of an array with a divide-and-conquer approach using OpenMP.
}
#define SIZE 1000000
int main() {
   // assume that SIZE, CUTOFF, and the number of OpenMP threads are reasonably specified, and nested parallelism is enabled.
   int a[SIZE]; int i; int CUTOFF=1000;
   for (i = 0; i<SIZE; ++i) { a[i] = rand() % SIZE ; } // initialization of input array 'a' with random integer numbers.
   int sum = parallel_sum(a, SIZE, CUTOFF);
   printf("parallel_sum = %d\n",sum);
   return 0; }
```